

Frontend Testing

Modul: M324

Autor: Metehan Celik

Setup

Vor dem Schreiben der Tests wurde die Testumgebung eingerichtet. Folgende Pakete wurden installiert:

```
npm install --save-dev jest react-test-renderer @testing-library/react @testing-library/jest-dom jest-environment-jsdom babel-jest @babel/preset-env @babel/preset-react jest-fetch-mock
```

Die Tests werden im Watch-Mode mit folgendem Befehl gestartet:

```
npm test
```

Dadurch laufen alle Tests automatisch neu, sobald eine Datei gespeichert wird.

Implementierte Tests

Die Tests prüfen die wichtigsten Methoden der App-Komponente: das Rendern, Hinzufügen, Löschen und Bearbeiten von Aufgaben.

Test 1 – Rendern der Überschrift

Prüft ob die ToDo-Liste den Titel korrekt anzeigt.

```
test('renders heading', () => {  
  render(<App />);  
  const headingElement = screen.getByRole('heading', { name: /ToDo Liste/i });  
  expect(headingElement).toBeInTheDocument();  
});
```

Test 2 – Aufgabe hinzufügen

Prüft ob über das Eingabefeld und den "Hinzufügen"-Button eine neue Aufgabe in der Liste erscheint.

```
test('allows user to add a new task', () => {  
  render(<App />);  
  const inputElement = screen.getByLabelText(/Neue Aufgabe hinzufügen/i);  
  const addButtonElement = screen.getByRole('button', { name: /Hinzufügen/i });
```

```

    fireEvent.change(inputElement, { target: { value: 'Buy groceries' } });
    fireEvent.click(addButtonElement);
    expect(screen.getByText('Buy groceries')).toBeInTheDocument();
  });

```

Test 3 – Aufgabe löschen

Prüft ob eine Aufgabe nach Klick auf den Löschen-Button verschwindet.

```

test('allows user to delete a task', () => {
  render(<App />);
  // Task hinzufügen
  fireEvent.change(screen.getByLabelText(/Neue Aufgabe hinzufügen/i), { target: {
    value: 'Putzen' } });
  fireEvent.click(screen.getByRole('button', { name: /Hinzufügen/i }));
  // Task löschen
  fireEvent.click(screen.getByRole('button', { name: /Löschen/i }));
  expect(screen.queryByText('Putzen')).not.toBeInTheDocument();
});

```

Test 4 – Aufgabe bearbeiten und speichern

Prüft den kompletten Edit-Flow: Bearbeiten-Button → Text ändern → Speichern.

```

test('allows user to edit and save a task', () => {
  render(<App />);
  fireEvent.change(screen.getByLabelText(/Neue Aufgabe hinzufügen/i), { target: {
    value: 'Kochen' } });
  fireEvent.click(screen.getByRole('button', { name: /Hinzufügen/i }));
  fireEvent.click(screen.getByRole('button', { name: /Bearbeiten/i }));
  fireEvent.change(screen.getByDisplayValue('Kochen'), { target: { value: 'Backen' } });
  fireEvent.click(screen.getByRole('button', { name: /Speichern/i }));
  expect(screen.getByText('Backen')).toBeInTheDocument();
});

```

p.test.js)

Übersicht aller Tests

#	Test	Geprüfte Methode
1	renders heading	Render
2	allows user to add a new task	handleSubmit
3	input field is cleared after adding a task	handleSubmit

4 does not add empty task	handleSubmit Validierung
5 can add multiple tasks	handleSubmit
6 allows user to delete a task	handleDelete
7 shows edit input when edit button is clicked	handleEdit
8 allows user to edit and save a task	handleSave
9 cancels edit mode without saving changes	Cancel-Button

Testausführung

Alle 9 Tests laufen mit einem einzigen Aufruf von `npm test` und sind grün.

```
/**
 * Testet das Rendern der Überschrift.
 */
test('renders heading', () => {
  render(<App />);
  const headingElement = screen.getByRole('heading', { name: /ToDo Liste/i });
  expect(headingElement).toBeInTheDocument();
});

/**
 * Testet das Hinzufügen einer Aufgabe.
 */
test('allows user to add a new task', () => {
  render(<App />);
  const inputElement = screen.getByLabelText(/Neue Aufgabe hinzufügen/i);
  const addButtonElement = screen.getByRole('button', { name: /Hinzufügen/i });
  const taskName = 'Buy groceries';
  fireEvent.change(inputElement, { target: { value: taskName } });
  fireEvent.click(addButtonElement);
  const newTaskElement = screen.getByText('Buy groceries');
  expect(newTaskElement).toBeInTheDocument();
});

/**
 * Testet, dass das Eingabefeld nach dem Hinzufügen geleert wird.
 */
test('input field is cleared after adding a task', () => {
  render(<App />);
  const inputElement = screen.getByLabelText(/Neue Aufgabe hinzufügen/i);
  const addButtonElement = screen.getByRole('button', { name: /Hinzufügen/i });
  fireEvent.change(inputElement, { target: { value: 'Sport' } });
  fireEvent.click(addButtonElement);
  expect(inputElement.value).toBe('');
});
```

```
/**
 * Testet, dass leere Eingaben nicht als Task hinzugefügt werden.
 */
test('does not add empty task', () => {
  render(<App />);
  const addButtonElement = screen.getByRole('button', { name: /Hinzufügen/i });
  fireEvent.click(addButtonElement);
  const listItems = screen.queryAllByRole('listitem');
  expect(listItems.length).toBe(0);
});

/**
 * Testet, dass mehrere Aufgaben hinzugefügt werden können.
 */
test('can add multiple tasks', () => {
  render(<App />);
  const inputElement = screen.getByLabelText(/Neue Aufgabe hinzufügen/i);
  const addButtonElement = screen.getByRole('button', { name: /Hinzufügen/i });

  fireEvent.change(inputElement, { target: { value: 'Task 1' } });
  fireEvent.click(addButtonElement);
  fireEvent.change(inputElement, { target: { value: 'Task 2' } });
  fireEvent.click(addButtonElement);

  expect(screen.getByText(/Task 1/)).toBeInTheDocument();
  expect(screen.getByText(/Task 2/)).toBeInTheDocument();
});
```

```
/**
 * Testet das Löschen einer Aufgabe (handleDelete).
 */
test('allows user to delete a task', () => {
  render(<App />);
  const inputElement = screen.getByLabelText(/Neue Aufgabe hinzufügen/i);
  const addButtonElement = screen.getByRole('button', { name: /Hinzufügen/i });

  fireEvent.change(inputElement, { target: { value: 'Putzen' } });
  fireEvent.click(addButtonElement);
  expect(screen.getByText('Putzen')).toBeInTheDocument();

  const deleteButton = screen.getByRole('button', { name: /Löschen/i });
  fireEvent.click(deleteButton);

  expect(screen.queryByText('Putzen')).not.toBeInTheDocument();
});

/**
 * Testet den Wechsel in den Edit-Modus (handleEdit).
 */
test('shows edit input when edit button is clicked', () => {
  render(<App />);
  const inputElement = screen.getByLabelText(/Neue Aufgabe hinzufügen/i);
  const addButtonElement = screen.getByRole('button', { name: /Hinzufügen/i });

  fireEvent.change(inputElement, { target: { value: 'Lesen' } });
  fireEvent.click(addButtonElement);

  const editButton = screen.getByRole('button', { name: /Bearbeiten/i });
  fireEvent.click(editButton);

  expect(screen.getByDisplayValue('Lesen')).toBeInTheDocument();
  expect(screen.getByRole('button', { name: /Speichern/i })).toBeInTheDocument();
});
```

```

test('allows user to edit and save a task', () => {
  render(<App />);
  const inputElement = screen.getByLabelText(/Neue Aufgabe hinzufügen/i);
  const addButtonElement = screen.getByRole('button', { name: /Hinzufügen/i });

  fireEvent.change(inputElement, { target: { value: 'Kochen' } });
  fireEvent.click(addButtonElement);

  fireEvent.click(screen.getByRole('button', { name: /Bearbeiten/i }));

  const editInput = screen.getByDisplayValue('Kochen');
  fireEvent.change(editInput, { target: { value: 'Backen' } });
  fireEvent.click(screen.getByRole('button', { name: /Speichern/i }));

  expect(screen.getByText('Backen')).toBeInTheDocument();
  expect(screen.queryByText('Kochen')).not.toBeInTheDocument();
});

/**
 * Testet das Abbrechen des Bearbeitens (Abbrechen-Button).
 */
test('cancels edit mode without saving changes', () => {
  render(<App />);
  const inputElement = screen.getByLabelText(/Neue Aufgabe hinzufügen/i);
  const addButtonElement = screen.getByRole('button', { name: /Hinzufügen/i });

  fireEvent.change(inputElement, { target: { value: 'Einkaufen' } });
  fireEvent.click(addButtonElement);

  fireEvent.click(screen.getByRole('button', { name: /Bearbeiten/i }));
  fireEvent.click(screen.getByRole('button', { name: /Abbrechen/i }));

  expect(screen.getByText('Einkaufen')).toBeInTheDocument();
  expect(screen.queryByRole('button', { name: /Speichern/i })).not.toBeInTheDocument();
});

```

```

PS C:\Users\Metehan Celik\Desktop\M324-ToDo\M324_PROJEKT_TODOLIST\frontend> npm test
Test Suites: 1 passed, 1 total
Tests:       9 passed, 9 total
Snapshots:   0 total
Time:        3.161 s
Ran all test suites related to changed files.

```

Watch Usage

- > Press **a** to run all tests.
- > Press **f** to run only failed tests.
- > Press **p** to filter by a filename regex pattern.
- > Press **t** to filter by a test name regex pattern.
- > Press **q** to quit watch mode.
- > Press **Enter** to trigger a test run.

Applikation läuft fehlerfrei

Nach den Anpassungen wurde die Applikation gestartet, um sicherzustellen, dass keine Coding-Probleme entstanden sind:

Backend:

```
mvn spring-boot:run
```

Frontend:

```
npm start
```

```
PS C:\Users\Metehan_Celik\Desktop\M324-ToDo\M324_PROJEKT_TODOLIST\frontend> npm start
```

```
→ Local:   http://localhost:5173/  
→ Network: use --host to expose  
→ press h + enter to show help
```

